

T.P. Matlab - Projet à faire pour l'examen de TP de décembre 2025

Ce projet est à faire pour préparer l'examen de TP qui aura lieu lors de la dernière séance de TP, le lundi 15 décembre 2025. Pendant cet examen de TP, vous aurez à utiliser et adapter les fonctions et les scripts programmés et validés dans ce projet.

Partie 1 : Résolution d'équations, interpolation polynômiale, intégration numérique

Exercice 1. *Résolution d'équations*

Toutes les instructions nécessaires à cet exercice (en plus des fonctions) seront regroupées dans le fichier `script1.m` (les fonctions étant définies dans des fichiers séparés).

1) Soit la fonction g_5 définie par

$$g_5(x) = -\frac{x^3}{6} + \frac{3x}{2},$$

estimer numériquement l'ordre de convergence de la méthode des itérés ($x_{k+1} = g(x_k)$) vers le point-fixe $r = \sqrt{3}$ à partir de $x_0 = 2$.

2) Soit la fonction f définie dans $\mathbb{R}^2$ par

$$f(x) = \begin{pmatrix} x_1 x_2 - x_1 - \frac{x_2}{2} + \frac{1}{2} \\ x_1 x_2^2 - x_2^2 - \frac{x_1}{4} + \frac{1}{4} \end{pmatrix},$$

avec $x = (x_1, x_2)$. Déterminer les racines de l'équation $f(x) = 0$.

Nous définissons un quadrillage régulier du rectangle $[0; 1.2] \times [-0.6; 1.2]$ d'abscisses `x01=0 :pas :1.2` et d'ordonnées `x02=-0.6 :pas :1.2` avec `pas=0.01`. Pour chaque point initial `x0=[x01(i) ;x02(j)]` calculer vers quelle solution de $f(x) = 0$ converge la méthode de Newton et représenter le quadrillage coloré en fonction de cette convergence. Signaler l'emplacement des racines par un "o" sur la figure.

Exercice 2. *Interpolation polynômiale*

Toutes les instructions nécessaires à cet exercice (en plus des fonctions) seront regroupées dans le fichier `script2.m`.

Pour `n=32, 64, 128, 256, 512`, calculer le polynôme d'interpolation de Lagrange par morceaux de degré $k = 3$ sur l'intervalle $[-5, 5]$ aux points équidistants $x_i = -5 + i \frac{10}{n}$ pour la fonction $f_2(x) = \frac{1}{1+x^2}$.

Exercice 3. *Intégration numérique*

Toutes les instructions nécessaires à cet exercice (en plus des fonctions) seront regroupées dans le fichier `script3.m`.

1) Calculer numériquement l'ordre de convergence et la constante de convergence pour les méthodes d'intégration numérique composées des trapèzes et de Simpson lors du calcul de

$$\int_0^1 \cos x \, dx.$$

2) Calculer numériquement le degré, l'ordre et la constante de convergence de la méthode de Newton-Cotes pour $l = 4$ (Voir le sujet du TP 4) : on estimera numériquement l'ordre de convergence en approchant l'intégrale $I = \int_0^1 e^x \, dx$ et on affichera la courbe de convergence.

Partie 2 : Dynamique des populations

On note $N(t)$ le nombre de proies et $P(t)$ le nombre de prédateurs. Le modèle de Lotka-Volterra est régi par le système d'équations différentielles suivant

$$\begin{cases} N'(t) &= (a - bP(t))N(t), \\ P'(t) &= (-p + qN(t))P(t), \end{cases}$$

où les coefficients a, b, p, q sont des constantes positives. Nous prendrons $a = 3, b = 2, p = 2.5, q = 2$. Nous considérons le problème écrit sous la forme $y'(t) = f(t, y(t))$ avec $y(0) = y_0 = (N_0, P_0)$ et $t \in [t_0, t_f]$.

Nous allons le résoudre par la récurrence implicite suivante :

$$y_{k+1} = y_k + hf\left(t_k + \frac{h}{2}, \frac{y_k + y_{k+1}}{2}\right), \quad \text{pour } k = 0, n-1,$$

où $n \in \mathbb{N}^*, h = \frac{t_f - t_0}{n}$ est le pas de la discrétisation en temps, $t_k = t_0 + kh$ pour $0 \leq k \leq n$ et y_k est une approximation de $y(t_k)$. C'est-à-dire qu'à chaque étape il y a un point fixe à calculer. Le plus simple à programmer est la récurrence $u_{j+1} = \Phi(u_j)$ pour une application contractante. Ce qui donne ici pour le point fixe $y_1 = y_0 + hf(t_0 + \frac{h}{2}, \frac{y_0 + y_1}{2})$:

```
ti=t0+h/2;zn=y0;znpr=y0+h*f(ti,(y0+zn)/2)';nit=1;
while norm(znpr-zn,inf)>tol;
    zn=znpr;znpr=y0+h*f(ti,(y0+zn)/2)';nit=nit+1;
    if (nit > nitmax)
        disp('Nombre maximal d iterations atteint');return;
    end
end
y(1,:) = znpr;tt(1)=t0+h;
```

où `tol` est la précision demandée pour le point fixe et `nitmax` son nombre maximal d'itérations.

Exercice 4. Toutes les instructions nécessaires à la résolution de cet exercice seront mises dans le même script `script4.m` (sauf les définitions de fonctions qui doivent être dans des fichiers séparés).

1) Écrire la fonction, `function [tt,y]=metimp(t0,tf,n,y0,f,tol,nitmax)`, qui résout un problème $y'(t) = f(t, y(t))$ par cette méthode implicite.

2) Prendre $t_0 = 0$, $t_f = 16$, $N_0 = 1.5$, $n = 2000$, $\text{tol}=1\text{e-}10$, $\text{nitmax}=100$, puis résoudre le système et tracer sur une même figure le portrait de phase (P en fonction de N) pour $P_0=2 : 1 : 5$.

3) Estimer numériquement à 10^{-2} près la période de chacune des courbes tracées à la question précédente.

Partie 3 : Méthodes itératives de résolution de systèmes linéaires

Exercice 5. Méthode de Relaxation pour les matrices pleines

Mettre toutes les instructions nécessaires à cet exercice (en plus des fonctions) dans le fichier `script5.m`.

1) Écrire une fonction `[x,nit,vecte]=IterRel(A,b,omega,x0,epsil,nitmax)` (respectivement `[x,nit,vecte]=IterJac(A,b,x0,epsil,nitmax)`) qui calcule la solution x du système linéaire $Ax = b$ par la méthode de relaxation (respectivement Jacobi) (avec l'algorithme de Richardson) à partir de l'initialisation x_0 , avec la précision `epsil`, le paramètre `omega` et le nombre maximal d'itérations `nitmax`. `nit` est le nombre d'itérations à la convergence et `vecte` est un vecteur contenant la norme 2 des résidus successifs.

2) Tester la fonction `IterRel` avec les données

$$A = \begin{pmatrix} 5 & -2 & 1 \\ -2 & 8 & -1 \\ 1 & -1 & 9 \end{pmatrix}, \quad b = \begin{pmatrix} -8 \\ 17 \\ 6 \end{pmatrix}, \quad x^{(0)} = \begin{pmatrix} 10 \\ 10 \\ 10 \end{pmatrix}.$$

$\omega = 1.0$, $\varepsilon = 0.000001$ et au maximum 100 itérations.

3) Calculer $\log_{10}(\rho(\mathcal{L}_\omega))$ où $\mathcal{L}_\omega$ est la matrice d'itération de la méthode de relaxation.

4) Nous changeons maintenant de données. Soit A la matrice carrée d'ordre n tridiagonale symétrique définie par :

$$a_{i,i} = 2, \quad a_{i,i-1} = a_{i,i+1} = -1, \quad a_{i,j} = 0 \text{ sinon,}$$

quant au second membre noté $b \in \mathbb{R}^n$ du système linéaire $Ax = b$, il est défini par :

$$b_1 = 1, \quad b_n = 1, \quad b_i = 0 \text{ pour } 2 \leq i \leq n-1.$$

Pour toutes les valeurs de ω de 0.1 à 1.9 par pas de 0.1 et $n = 10$, calculer la solution pour les données ci-dessus, avec le vecteur initial $x^0 = 0$, $\varepsilon = 10^{-6}$, au maximum 200 itérations. Tracer la courbe $\rho(\mathcal{L}_\omega)$ en fonction de ω .

5) Pour n prenant les valeurs 16, 32, 64, 128, 256, 512, au maximum 2000 itérations, représenter sur une figure le $\log_{10}$ des temps de résolution en fonction de $\log_{10}(n)$ pour les méthodes de Jacobi, Gauss-Seidel et Relaxation.

Exercice 6. Méthode de Relaxation pour les matrices tridiagonales

Mettre toutes les instructions nécessaires à cet exercice (en plus des fonctions) dans le fichier `script6.m`.

Nous considérons ici la matrice tridiagonale A d'ordre n et le second membre b définissant le système linéaire $Ax = b$ de l'exercice précédent.

1) Écrire une fonction

`[x,nit,vecte]=IterReltridiag(d,l,u,b,omega,x0,epsil,nitmax)` qui calcule la solution x du système linéaire $Ax = b$ pour une matrice A tridiagonale (d est la diagonale, l la première sous-diagonale et u la première sur-diagonale de A) par la méthode de relaxation (avec l'algorithme de Richardson) à partir de l'initialisation x_0 , avec la précision `epsil`, le paramètre `omega` et le nombre maximal d'itérations `nitmax`. `nit` est le nombre d'itérations à la convergence et `vecte` est un vecteur contenant la norme 2 des résidus successifs.

2) Pour n prenant les valeurs 16, 32, 64, 128, 256, 512, au maximum 2000 itérations, représenter sur une même figure le $\log_{10}$ des temps de résolution en fonction de $\log_{10}(n)$ pour les 2 méthodes : `IterRel` et `IterReltridiag` avec le paramètre optimal ω_{opt} , $x^{(0)} = 0$, $\varepsilon = 10^{-6}$.